

Predicting Hits with SVM: 2026 Balls in Play

Goal

The goal of this project was to create a predictive model using launch angle and exit velocity from every batted ball in play during the 2026 MLB season to classify whether each ball in play resulted in a hit or an out. I used a support vector machine (SVM) through the `ksvm` function in R, the same approach used in ISYE 6501 homework, and applied it to real Statcast data to see how much two simple measurements can tell you about the outcome of a batted ball.

Methodology

1. A CSV containing every batted ball from the 2026 regular season (March through September) was downloaded from Baseball Savant and read into R using `read.csv` with `header = TRUE`. The `kernlab` library was loaded to use the `ksvm` function.
2. The data was cleaned so that only balls in play with usable tracking data were included.
 - a. Two TRUE/FALSE checks, `has_LS` and `has_LA`, were created to flag rows where `launch_speed` and `launch_angle` were not missing.
 - b. Only rows that passed both checks were kept and saved as `bip`.
3. The response variable, `hit`, was created.
 - a. Using the `events` column, a ball in play was marked as a hit if the outcome was a single, double, triple, or home run.
 - b. `ifelse` was used to convert that TRUE/FALSE column into 1 for a hit and 0 for an out.
 - c. `table()` was run on the `hit` column to confirm every row was labeled and to find the baseline, which is the accuracy a model would get by predicting an out every single time.
4. Since SVMs slow down a lot as the data gets bigger, a random sample of 10,000 balls in play was taken to train the model on.
 - a. `set.seed(42)` was placed right before `sample()` so the same sample would come out every time the script was rerun.
 - b. `sample(nrow(bip), 10000)` picked 10,000 random row numbers, which were saved as `idx` and used to pull those rows out of `bip` as `bip_sample`.
 - c. `table()` was run on the `hit` column of `bip_sample` to make sure the hit and out split matched the full data.
5. A first model was built using a linear kernel (`vanilladot`) with `C = 100`, using `launch_angle` and `launch_speed` as predictors and `hit` as the response. The predictors were wrapped in `as.matrix()` and the response in `as.factor()`, the same way as in the homework.

6. Because the linear model could not beat the baseline, the kernel was switched to rbfdot, which allows the boundary between hits and outs to bend instead of being a straight line.
7. A test set was created out of every ball in play the model did not train on.
 - a. `bip[-idx,]` grabbed every row except the 10,000 in the sample, and it was saved as `bip_test`.
 - b. This gave 110,879 balls in play the model had never seen.
8. A range of C values was tested on a log scale: 0.01, 0.1, 1, 100, and 1000. For each C value:
 - a. `set.seed(42)` was run directly before the model line so rbfdot would give the same result every time.
 - b. The model was trained on `bip_sample`.
 - c. `predict` was run on `bip_sample` to calculate training accuracy.
 - d. `predict` was run on `bip_test` to calculate test accuracy.
 - e. `table()` was run comparing `pred_test` to the actual hit column of `bip_test` to get a confusion matrix showing what kind of mistakes the model made.
 - f. Exit velocity was plotted against launch angle for both the sample and the test set, with each ball colored red if the model predicted a hit and gray if it predicted an out.
9. The actual outcomes for the sample and test set were also plotted using the same layout, so the predicted plots could be compared directly against what really happened.

Results

Baseline. Of the 120,879 balls in play in the full data, 81,612 were outs, or about 67.5%. The 10,000 ball sample matched that closely, with 6,787 outs (67.9%). That means a model that just predicts an out every time would already be right about 67.5% of the time, so any model has to beat that to be worth anything.

```
> table(bip$hit)
 0     1
81612 39267
> table(bip_sample$hit)
 0     1
6787 3213
```

Linear model (vanilladot). The linear model predicted an out for all 10,000 balls in play in the sample, giving an accuracy of 0.6787, which is exactly the out rate in the sample. A straight line could not separate hits from outs any better than just guessing out every time.

```

> set.seed(42)
> model_lin <- ksvm(as.matrix(bip_sample[,c("launch_angle","launch_speed")]),
+                  as.factor(bip_sample$hit), type="C-svc", kernel="vanilladot",
+                  C=100, scaled=TRUE)
Setting default kernel parameters
> pred_lin <- predict(model_lin, bip_sample[,c("launch_angle","launch_speed")])
> sum(pred_lin == bip_sample$hit) / nrow(bip_sample)
[1] 0.6787
> table(pred_lin)
pred_lin
      0      1
10000      0

```

rbfdot model across C values. Switching to rbfdot let the model actually predict hits. Training accuracy and the number of hits and outs predicted for each C value are shown below, in order from C = 1000 down to C = 0.01.

Sample

```

> set.seed(42)
> model <- ksvm(as.matrix(bip_sample[,c("launch_angle","launch_speed")]),
+              as.factor(bip_sample$hit), type="C-svc", kernel="rbfdot",
+              C=1000, scaled=TRUE)
>
> pred <- predict(model, bip_sample[,c("launch_angle","launch_speed")])
>
> sum(pred == bip_sample$hit) / nrow(bip_sample)
[1] 0.8044
> table(pred)
pred
  0    1
7045 2955

```

```

> set.seed(42)
> model <- ksvm(as.matrix(bip_sample[,c("launch_angle","launch_speed")]),
+              as.factor(bip_sample$hit), type="C-svc", kernel="rbfdot",
+              C=100, scaled=TRUE)
>
> pred <- predict(model, bip_sample[,c("launch_angle","launch_speed")])
>
> sum(pred == bip_sample$hit) / nrow(bip_sample)
[1] 0.8017
> table(pred)
pred
  0    1
6990 3010

```

```
> set.seed(42)
> model <- ksvm(as.matrix(bip_sample[,c("launch_angle","launch_speed")] ),
+               as.factor(bip_sample$hit),type="C-svc",kernel="rbfdot",
+               C=1,scaled=TRUE)
>
> pred <- predict(model,bip_sample[,c("launch_angle","launch_speed")])
>
> sum(pred == bip_sample$hit) / nrow(bip_sample)
[1] 0.798
> table(pred)
pred
  0    1
6729 3271
```

```
> set.seed(42)
> model <- ksvm(as.matrix(bip_sample[,c("launch_angle","launch_speed")] ),
+               as.factor(bip_sample$hit),type="C-svc",kernel="rbfdot",
+               C=.1,scaled=TRUE)
>
> pred <- predict(model,bip_sample[,c("launch_angle","launch_speed")])
>
> sum(pred == bip_sample$hit) / nrow(bip_sample)
[1] 0.7936
> table(pred)
pred
  0    1
6531 3469
```

```
> set.seed(42)
> model <- ksvm(as.matrix(bip_sample[,c("launch_angle","launch_speed")] ),
+               as.factor(bip_sample$hit),type="C-svc",kernel="rbfdot",
+               C=.01,scaled=TRUE)
>
> pred <- predict(model,bip_sample[,c("launch_angle","launch_speed")])
>
> sum(pred == bip_sample$hit) / nrow(bip_sample)
[1] 0.758
> table(pred)
pred
  0    1
7749 2251
```

Test accuracy and the confusion matrix for each C value are shown below, in the same order.

Test

```
> set.seed(42)
>
> model <- ksvm(as.matrix(bip_sample[,c("launch_angle","launch_speed")] ),
+              as.factor(bip_sample$hit),type="C-svc",kernel="rbfdot",
+              C=1000,scaled=TRUE)
>
> pred_test <- predict(model,bip_test[,c("launch_angle","launch_speed")])
>
> sum(pred_test == bip_test$hit) / nrow(bip_test)
[1] 0.8064196
>
> table(pred_test, bip_test$hit)

pred_test      0      1
      0 65432 12071
      1  9393 23983
```

```
> set.seed(42)
>
> model <- ksvm(as.matrix(bip_sample[,c("launch_angle","launch_speed")] ),
+              as.factor(bip_sample$hit),type="C-svc",kernel="rbfdot",
+              C=100,scaled=TRUE)
>
> pred_test <- predict(model,bip_test[,c("launch_angle","launch_speed")])
>
> sum(pred_test == bip_test$hit) / nrow(bip_test)
[1] 0.8048954
>
> table(pred_test, bip_test$hit)

pred_test      0      1
      0 65019 11827
      1  9806 24227
```

```

> set.seed(42)
>
> model <- ksvm(as.matrix(bip_sample[,c("launch_angle","launch_speed")] ),
+               as.factor(bip_sample$hit),type="C-svc",kernel="rbfdot",
+               C=1,scaled=TRUE)
>
> pred_test <- predict(model,bip_test[,c("launch_angle","launch_speed")])
>
> sum(pred_test == bip_test$hit) / nrow(bip_test)
[1] 0.7993939
>
> table(pred_test, bip_test$hit)

pred_test      0      1
      0 63229 10647
      1 11596 25407

```

```

> set.seed(42)
>
> model <- ksvm(as.matrix(bip_sample[,c("launch_angle","launch_speed")] ),
+               as.factor(bip_sample$hit),type="C-svc",kernel="rbfdot",
+               C=.1,scaled=TRUE)
>
> pred_test <- predict(model,bip_test[,c("launch_angle","launch_speed")])
>
> sum(pred_test == bip_test$hit) / nrow(bip_test)
[1] 0.7928192
>
> table(pred_test, bip_test$hit)

pred_test      0      1
      0 61666  9813
      1 13159 26241

```

```

> set.seed(42)
>
> model <- ksvm(as.matrix(bip_sample[,c("launch_angle","launch_speed")] ),
+               as.factor(bip_sample$hit),type="C-svc",kernel="rbfdot",
+               C=.01,scaled=TRUE)
>
> pred_test <- predict(model,bip_test[,c("launch_angle","launch_speed")])
>
> sum(pred_test == bip_test$hit) / nrow(bip_test)
[1] 0.7583312
>
> table(pred_test, bip_test$hit)

pred_test      0      1
      0 66694 18665
      1  8131 17389

```

Training and test accuracy for each C value:

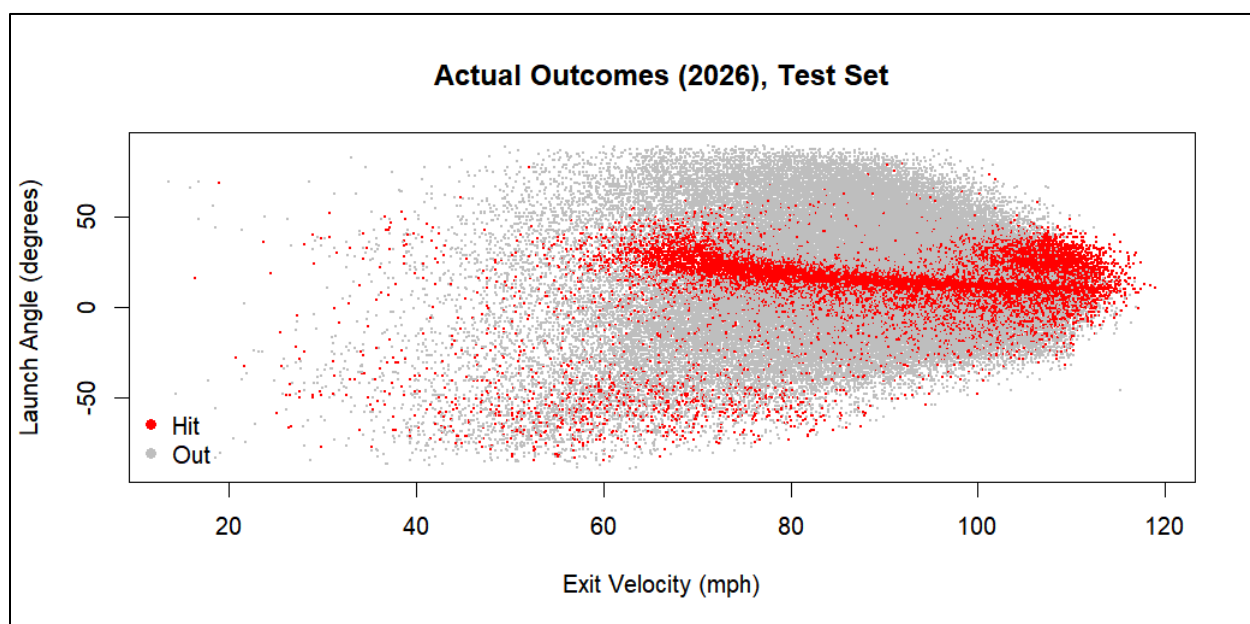
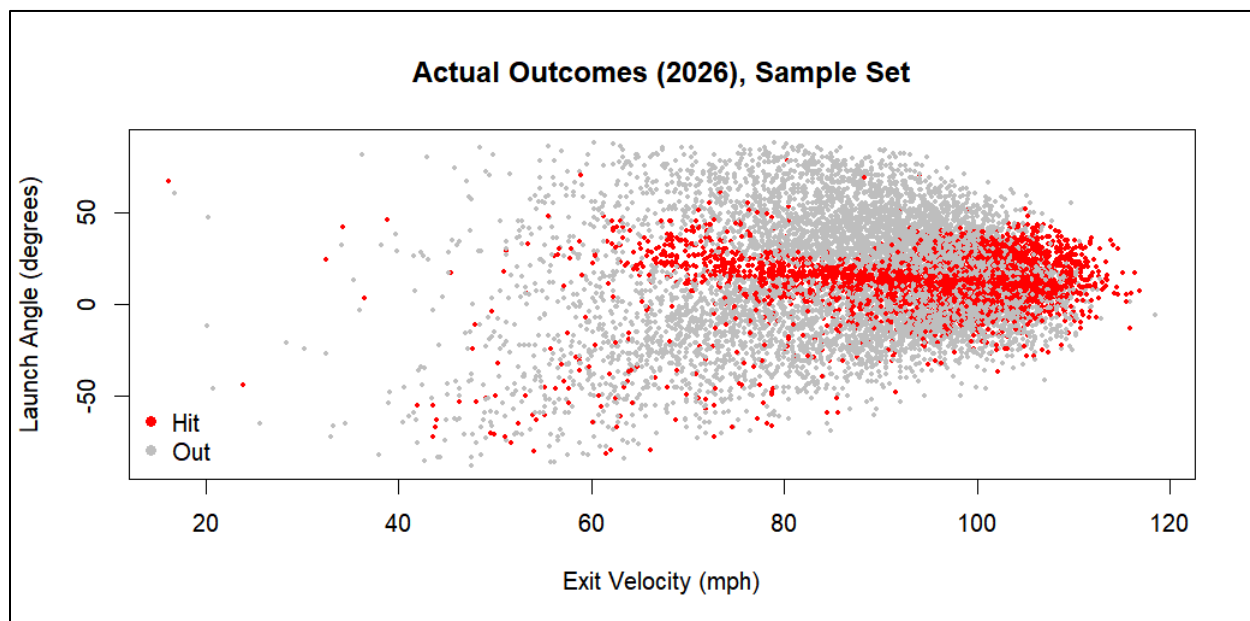
C	Training accuracy	Test accuracy
0.01	0.7580	0.7583
0.1	0.7936	0.7928
1	0.7980	0.7994
100	0.8017	0.8049
1000	0.8044	0.8064

Confusion matrix results on the test set (110,879 balls in play):

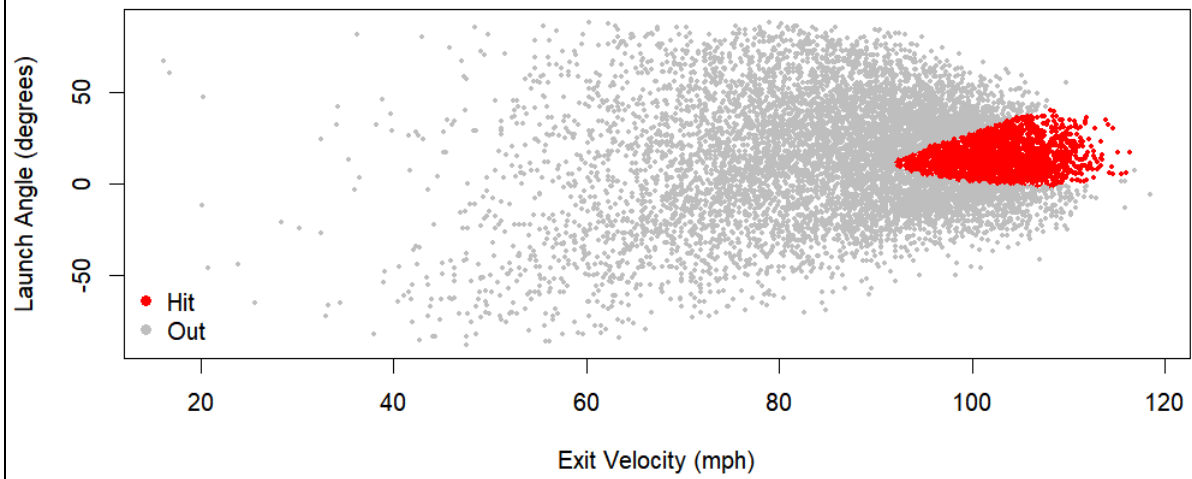
C	Correct outs	Missed hits	False hits	Correct hits
0.01	66,694	18,665	8,131	17,389
0.1	61,666	9,813	13,159	26,241
1	63,229	10,647	11,596	25,407
100	65,019	11,827	9,806	24,227
1000	65,432	12,071	9,393	23,983

The same confusion matrices as rates:

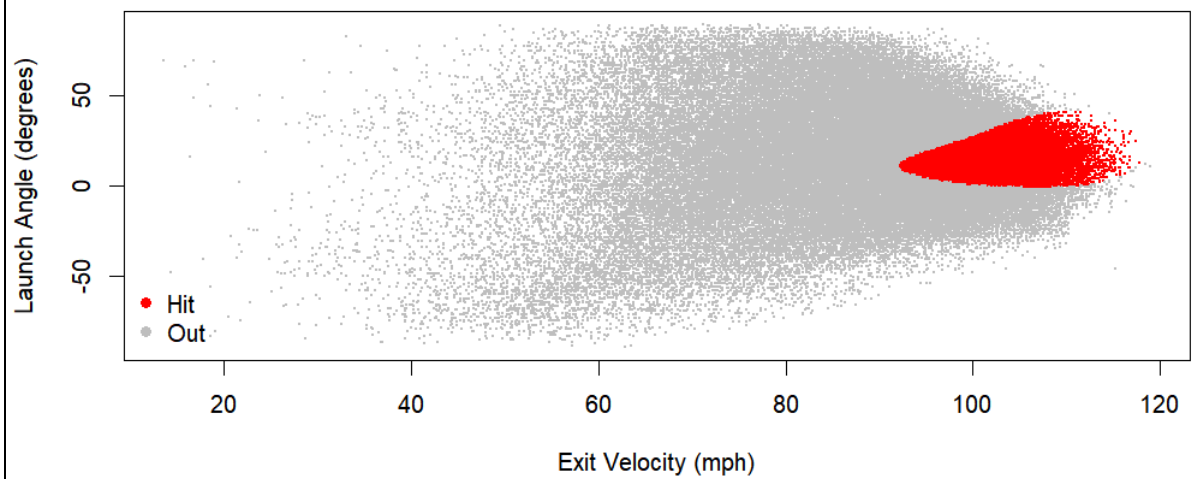
C	Hits caught	Right when it predicts a hit	Outs correct
0.01	48%	68%	89%
0.1	73%	67%	82%
1	70%	69%	85%
100	67%	71%	87%
1000	67%	72%	87%



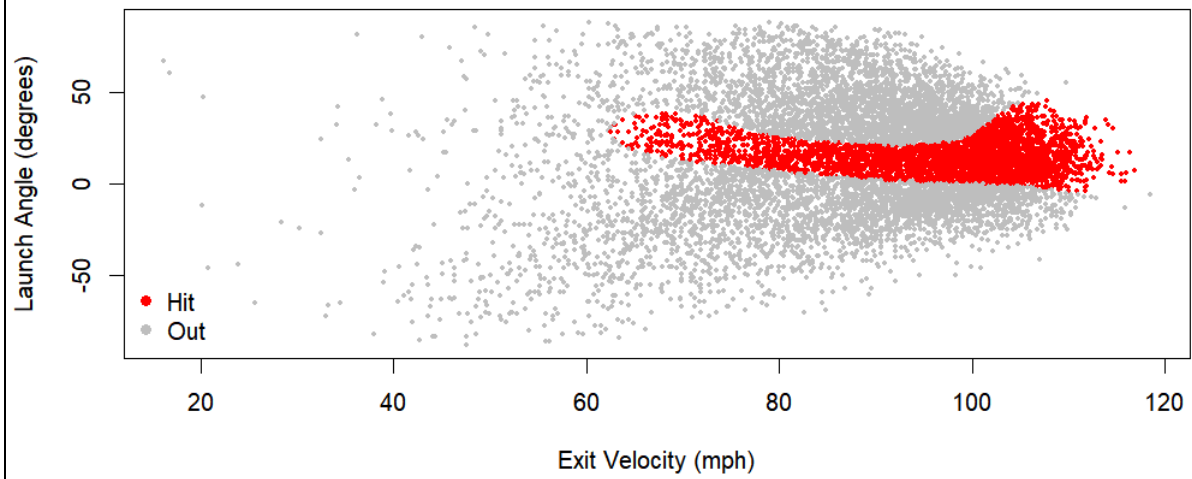
SVM Predicted Outcomes (2026), Sample Set (C=.01)



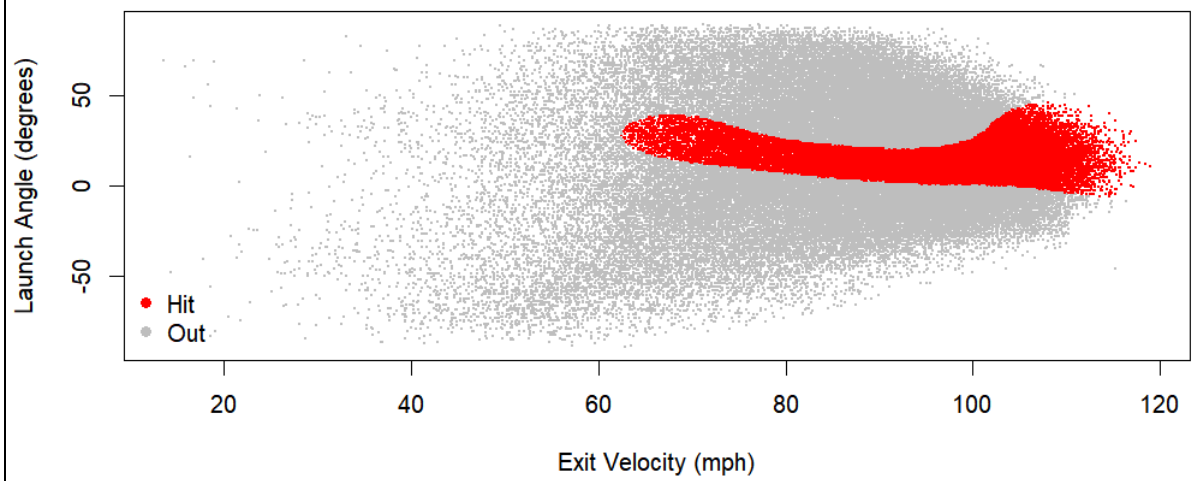
SVM Predicted Outcomes (2026), Test Set (C=.01)



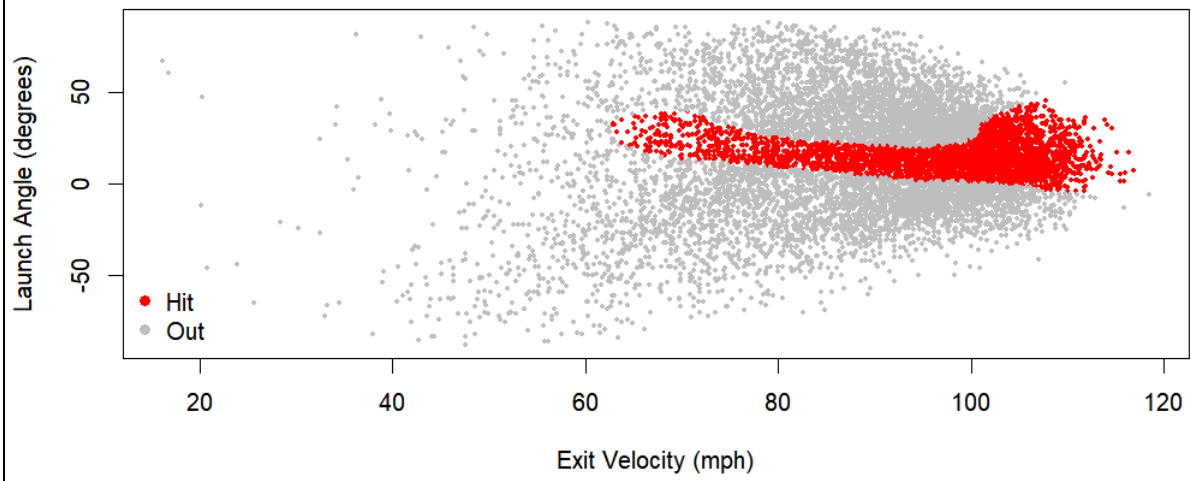
SVM Predicted Outcomes (2026), Sample Set (C=.1)



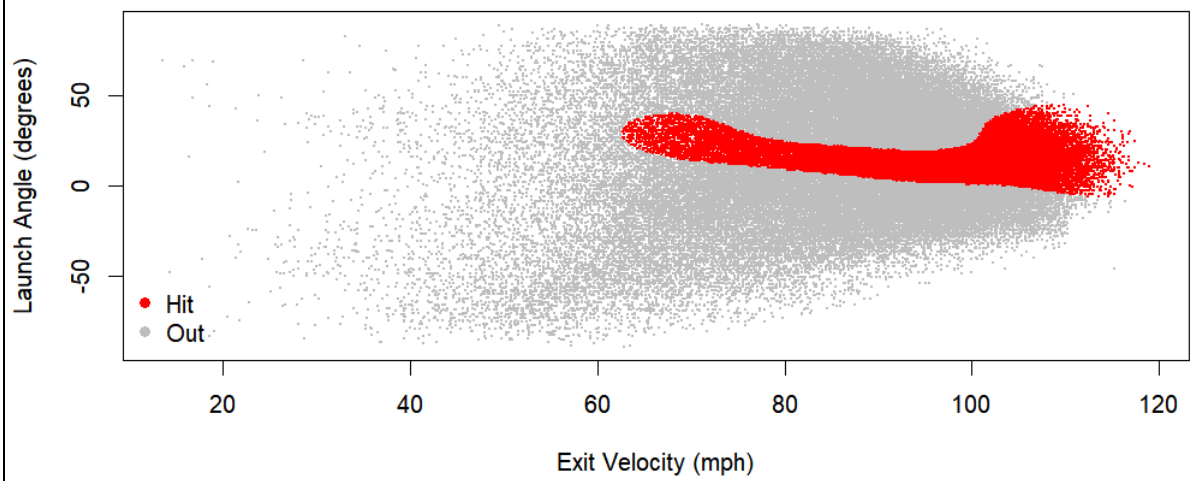
SVM Predicted Outcomes (2026), Test Set (C=.1)



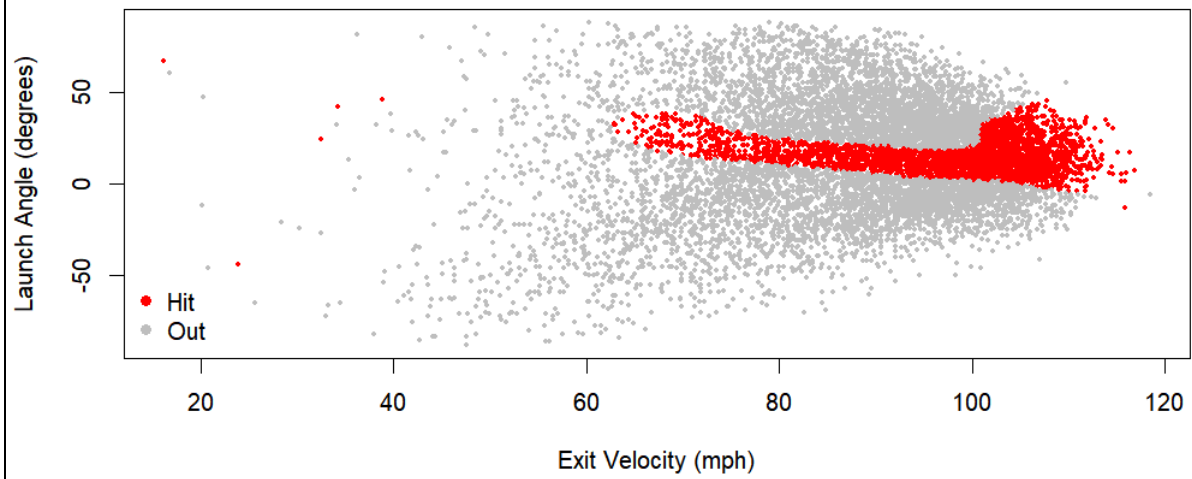
SVM Predicted Outcomes (2026), Sample Set (C=1)



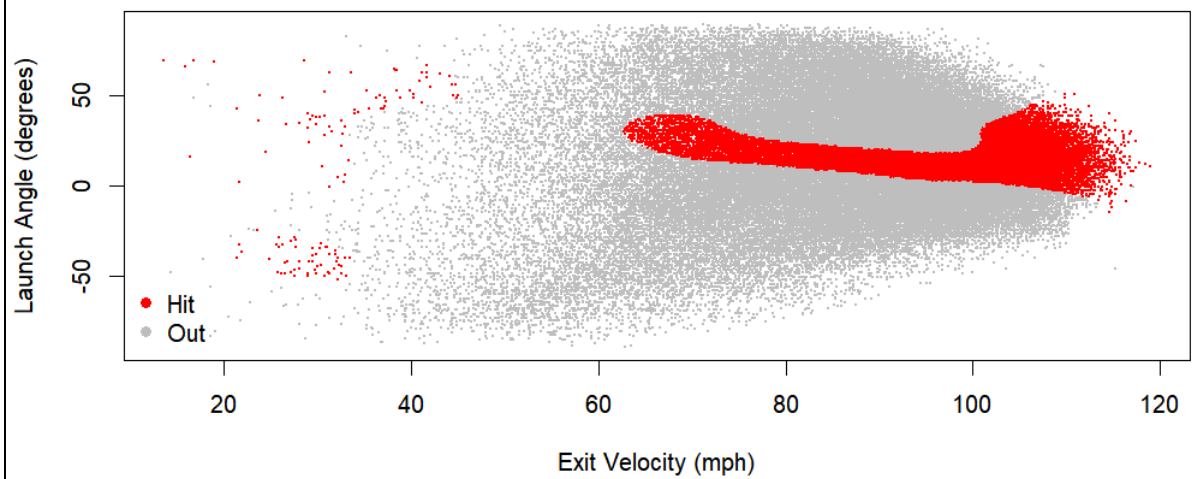
SVM Predicted Outcomes (2026), Test Set (C=1)

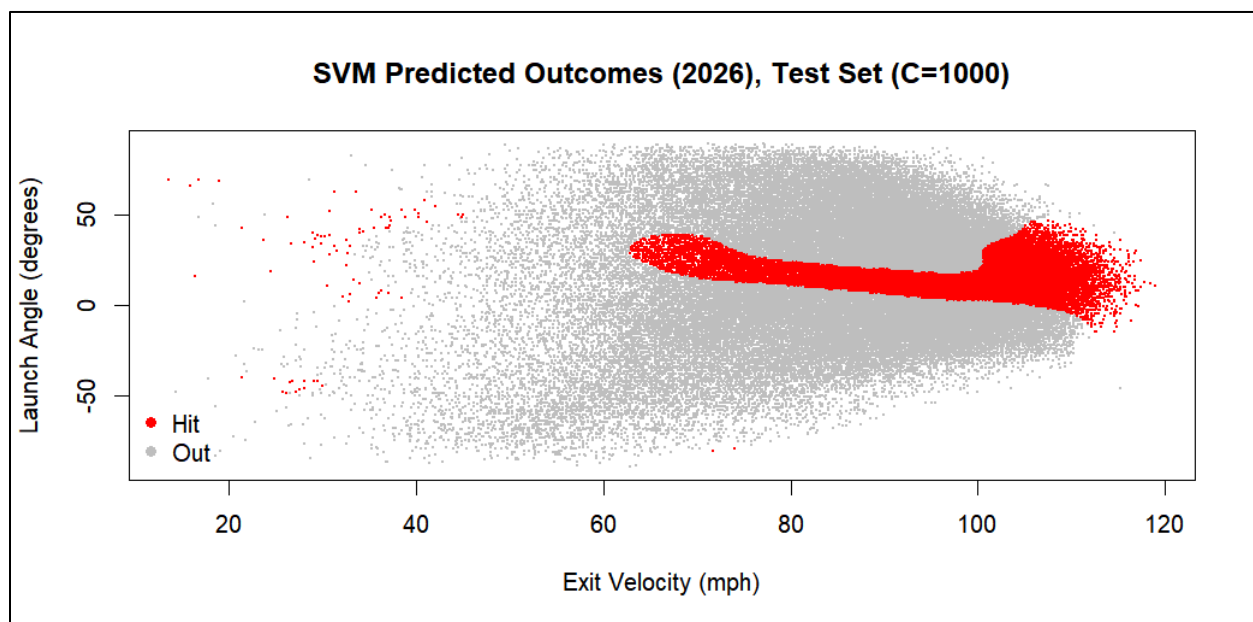
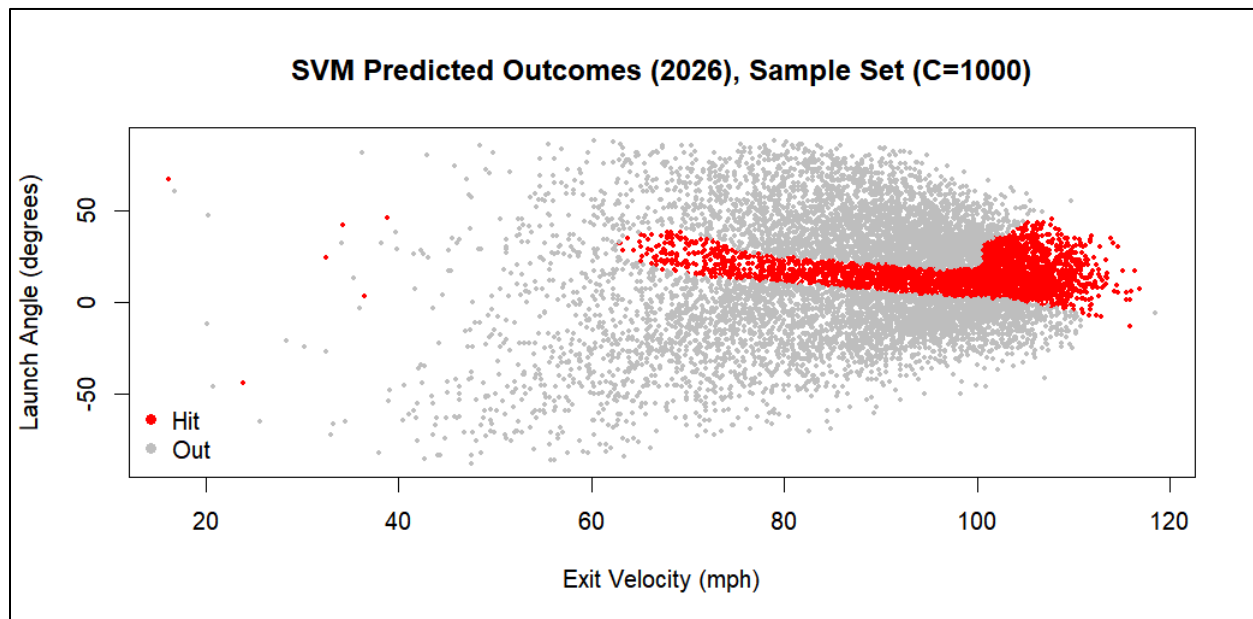


SVM Predicted Outcomes (2026), Sample Set (C=100)



SVM Predicted Outcomes (2026), Test Set (C=100)





Discussion

The biggest takeaway is that launch angle and exit velocity alone can predict whether a ball in play is a hit about 80% of the time, compared to a 67.5% baseline, and that result held up on over 110,000 balls in play the model never saw. The linear model could not beat the baseline at all because hits don't sit on one side of a line. They live in a pocket in the middle: line drives at medium launch angles, plus a separate block of hard hit balls past about 100 mph that turn into home runs. Once rbfdot was allowed to bend the boundary, the model found that shape on its own, and you can see the line drive stripe and the hard contact block clearly in the predicted plots.

The C values showed both ends of the tradeoff. At $C = 0.01$ the model was too simple and underfit. The whole line drive stripe disappeared and it only trusted the really hard hit balls, so it caught just 48% of real hits. As C went up, accuracy kept improving but the gains got smaller each step, from about 3.5 points between 0.01 and 0.1 down to only 0.15 points between 100 and 1000. At high C values the model also started drawing small islands of predicted hits at low exit velocity. Those are spots where it memorized specific training points, and they don't reflect real baseball logic. They barely cost any accuracy because so few balls land there, but they show the cost of letting the model get too flexible. Across every C value the gap between training and test accuracy stayed tiny, so overfitting was never a big problem overall with this much data.

$C = 1000$ had the highest test accuracy at 0.8064, but it was basically tied with $C = 100$ at 0.8049, a difference of about 165 balls out of 110,879. I would argue $C = 1$ is also a strong choice since it gives up less than a point of accuracy and draws a clean boundary with no islands. Overall accuracy also isn't the only way to judge the model. $C = 0.1$ caught the most real hits (73%) but was the least precise when it called one, while $C = 1000$ was the most precise (72%) but missed more hits. Which C is best depends on whether missing a hit or falsely calling one matters more.

Most of the model's remaining mistakes make sense from a baseball standpoint. The missed hits are mostly bloop, ground balls that find a hole, and infield singles, which look like outs based on exit velocity and launch angle. The false hits are most likely hard line drives hit right at a fielder. Those two variables can't tell where the ball went or how fast the runner is, so some of that error can't be fixed with this setup.

For next steps, I would add spray angle and sprint speed as predictors, since those cover a lot of what the model is currently missing. I would also switch to probability outputs instead of just hit or out, which would let me build my own expected batting average for each hitter and compare it to their actual batting average to see who got lucky or unlucky this season.